

Clustering am Beispiel von Geo-Daten

Contents

- Clustering der start stations
- DBSCAN
- Great Circle Distance
- Vergleich K -Means & DBSCAN

Nach Tutorial @ <https://chih-ling-hsu.github.io/2018/01/02/clustering-python#elbow-method>

```
import pandas as pd
import numpy as np
from os.path import join

from geopy.distance import vincenty
from sklearn.cluster import KMeans, DBSCAN
from sklearn.preprocessing import StandardScaler

import warnings
warnings.filterwarnings('ignore')

# visualization
from pandas.plotting import scatter_matrix
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from mpl_toolkits.basemap import Basemap

plt.style.use('ggplot')
%matplotlib inline
import seaborn as sns
sns.set(color_codes=True)
sns.set(style="darkgrid")
sns.set_context("paper", font_scale=1.5)
```

```
# Some helper functions
def plot_stations_map(ax, stns, cluster_col, lat_col, lon_col):
    # determine range to print based on min, max lat and lon of the data
    lat = list(stns[lat_col])
    lon = list(stns[lon_col])
    margin = 0.01 # buffer to add to the range
    lat_min = min(lat) - margin
    lat_max = max(lat) + margin
    lon_min = min(lon) - margin
    lon_max = max(lon) + margin

    # create map using BASEMAP
    m = Basemap(llcrnrlon=lon_min,
                llcrnrlat=lat_min,
                urcrnrlon=lon_max,
                urcrnrlat=lat_max,
                lat_0=(lat_max - lat_min)/2,
                lon_0=(lon_max - lon_min)/2,
                projection='lcc',
                resolution = 'f',)

    m.drawcoastlines()
    m.fillcontinents(lake_color='aqua')
    m.drawmapboundary(fill_color='aqua')
    m.drawrivers()

    # plot points
    clist = list(stns[cluster_col].unique())
    if -1 in clist:
        clist.remove(-1)
    k = len(clist)
    colors = iter(cm.Set1(np.linspace(0, 1, max(10, k))))
    for i in range(k):
        color = next(colors)
        df = stns.loc[stns[cluster_col] == clist[i]]
        #print("Cluster {} has {} samples.".format(clist[i], df.shape[0]))

        # convert lat and lon to map projection coordinates
        lons, lats = m(list(df[lon_col]), list(df[lat_col]))
        ax.scatter(lons, lats, marker = 'o', color=color, edgecolor='gray', zorder=5, alpha=1.0, s=1
```

```
# Elbow Method for finding 'optimal' K
# je größer k desto geringer die sum of squared distances (im Extremfall ist jeder Punkt ein Cluster)
def compute_sosd(data, k_range=range(1,10)):
    sum_of_squared_distances = []
    for k in k_range:
        km = KMeans(n_clusters=k, random_state=10)
        km = km.fit(data)
        sum_of_squared_distances.append(km.inertia_)
    return sum_of_squared_distances
```

```
data = pd.read_csv(join("data", "201707-citibike-tripdata.csv"))
```

```
data.head()
```

	tripduration	starttime	stoptime	start station id	start station name	start station latitude	start station longitude	end station id	end station name
0	364	2017-07-01 00:00:00	2017-07-01 00:06:05	539	Metropolitan Ave & Bedford Ave	40.715348	-73.960241	3107	Bedford Ave & Nassau Ave
1	2142	2017-07-01 00:00:03	2017-07-01 00:35:46	293	Lafayette St & E 8 St	40.730207	-73.991026	3425	2 Ave & E 104 St
2	328	2017-07-01 00:00:08	2017-07-01 00:05:37	3242	Schermerhorn St & Court St	40.691029	-73.991834	3397	Court St & Nelson St
3	2530	2017-07-01 00:00:11	2017-07-01 00:42:22	2002	Wythe Ave & Metropolitan Ave	40.716887	-73.963198	398	Atlantic Ave & Furman St
4	2534	2017-07-01 00:00:15	2017-07-01 00:42:29	2002	Wythe Ave & Metropolitan Ave	40.716887	-73.963198	398	Atlantic Ave & Furman St

```
# convert datetime columns to the right type
data['starttime'] = pd.to_datetime(data['starttime'])
data['stoptime'] = pd.to_datetime(data['stoptime'])
# extract day of week
data['starttime_dow'] = data['starttime'].map(lambda x: x.dayofweek) # Monday=0, Sunday=6
data['stoptime_dow'] = data['stoptime'].map(lambda x: x.dayofweek)
```

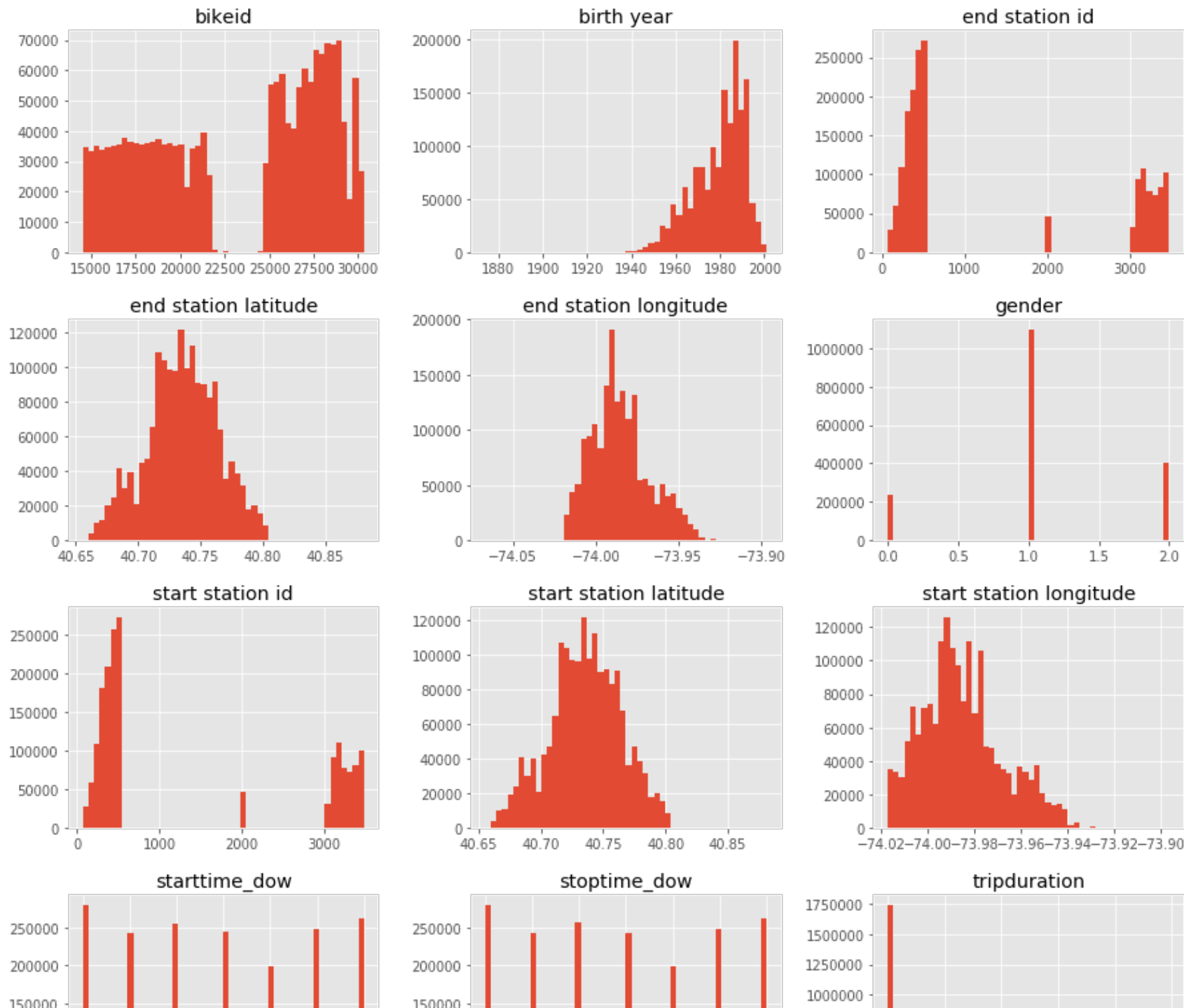
```
data.shape
```

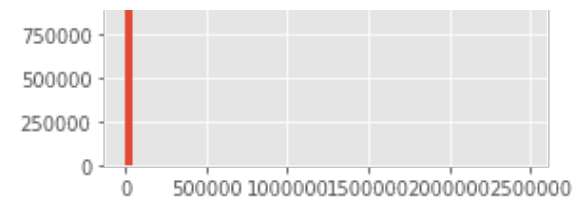
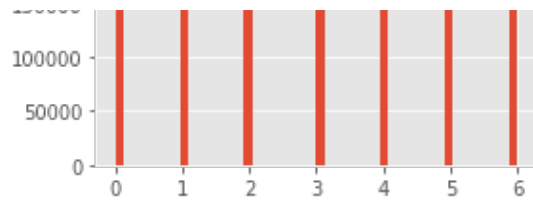
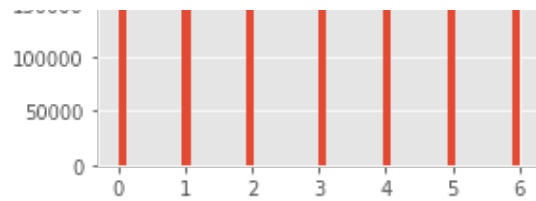
```
(1735599, 17)
```

```
data.columns
```

```
Index(['tripduration', 'starttime', 'stoptime', 'start station id',  
      'start station name', 'start station latitude',  
      'start station longitude', 'end station id', 'end station name',  
      'end station latitude', 'end station longitude', 'bikeid', 'usertype',  
      'birth year', 'gender', 'starttime_dow', 'stoptime_dow'],  
      dtype='object')
```

```
data.hist(bins=50, figsize=(15,15));
```





```
data.starttime.min()
```

```
Timestamp('2017-07-01 00:00:00')
```

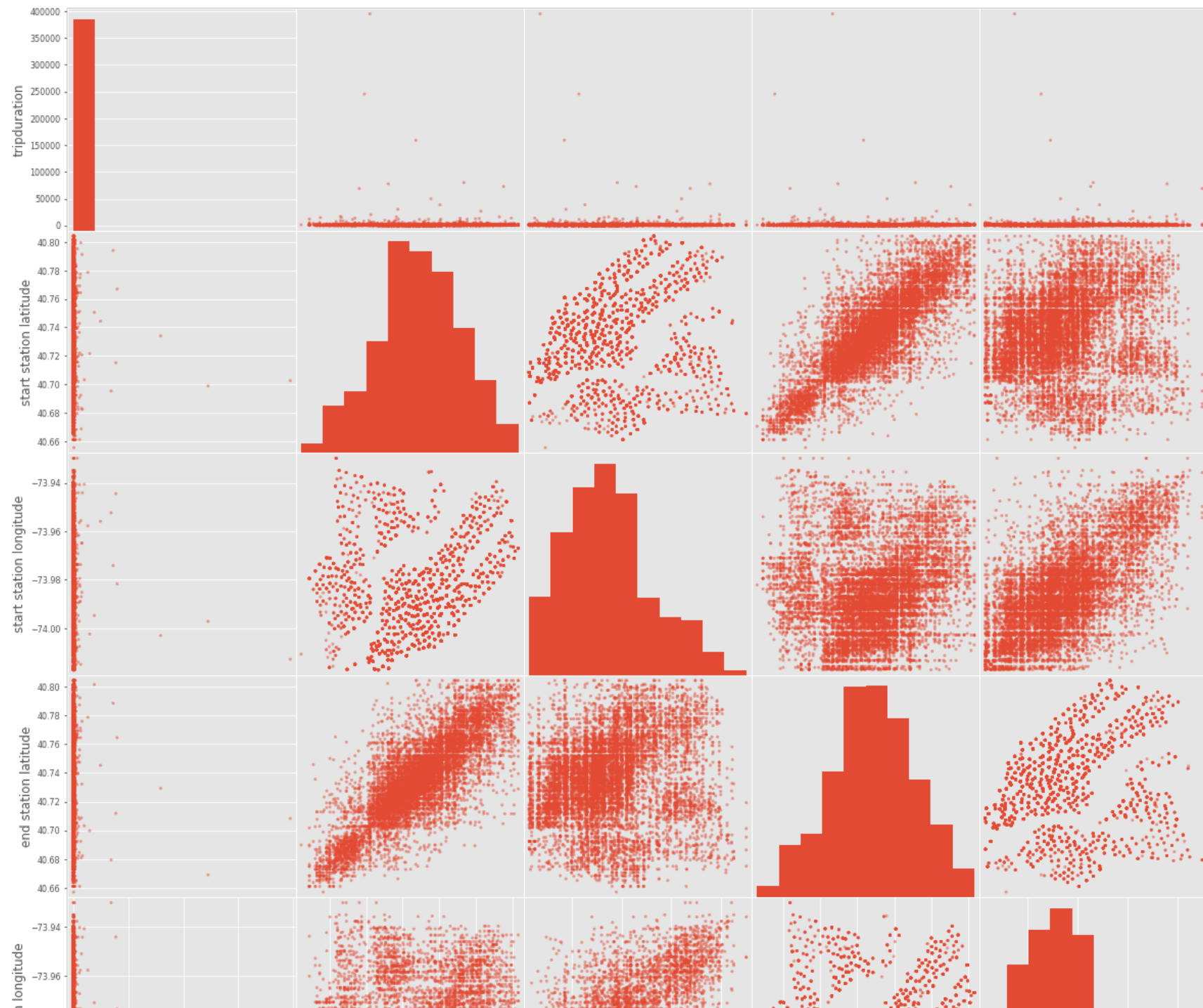
```
data.starttime.max()
```

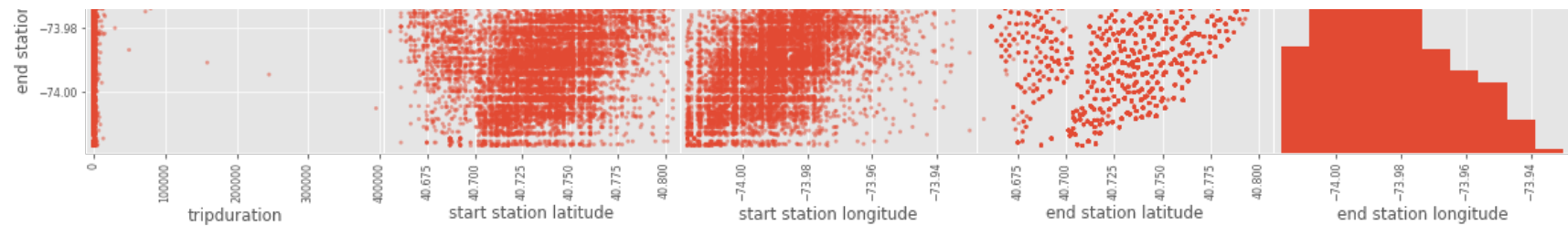
```
Timestamp('2017-07-31 23:59:57')
```

The data relates to the entire July 2017

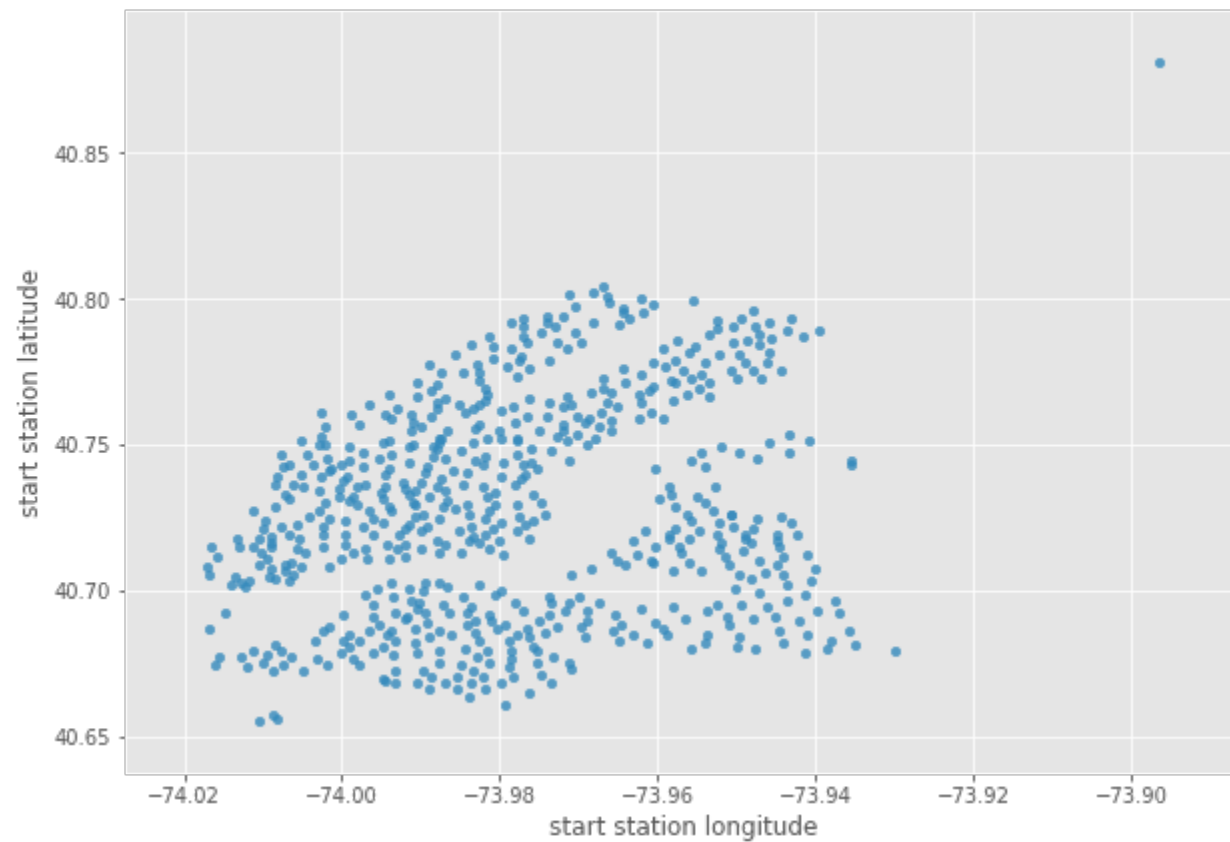
```
X = data[['start station latitude', 'start station longitude']].drop_duplicates()
```

```
cols = ['tripduration', 'starttime',
        'start station latitude', 'start station longitude',
        'end station latitude', 'end station longitude']
scatter_matrix(data[cols].sample(10000), figsize=(20, 20), alpha=0.5);
```





```
X.plot(kind="scatter", x="start station longitude", y="start station latitude", alpha=0.8,  
       figsize=(10,7));  
plt.legend();
```



Der Punkt rechts oben scheint ein Outlier zu sein, den wir aus den Daten entfernen:

```
data = data[data['start station latitude']<40.85].reset_index()
```

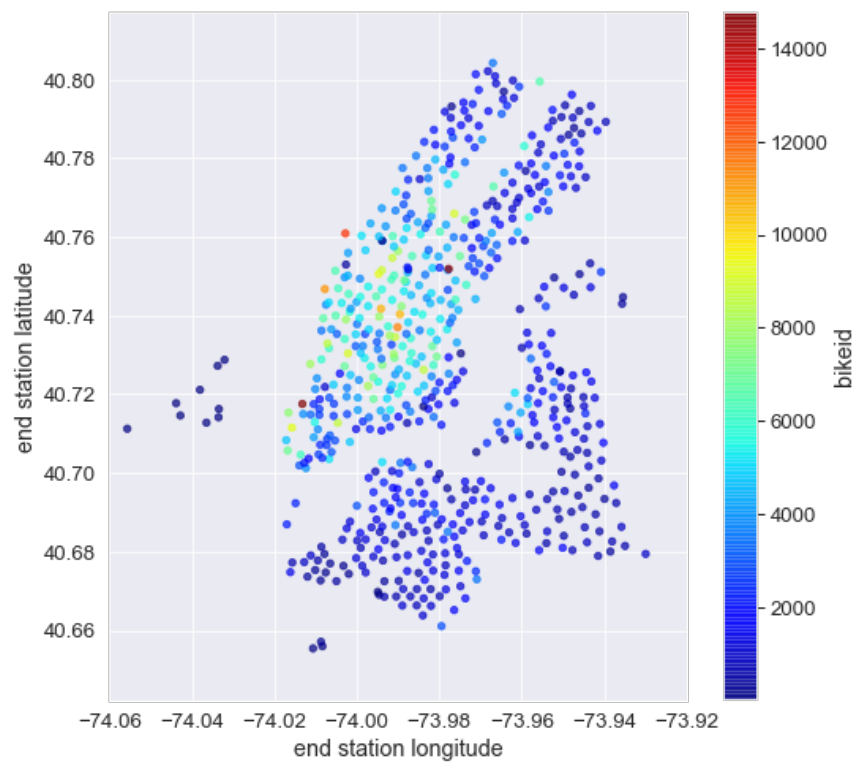
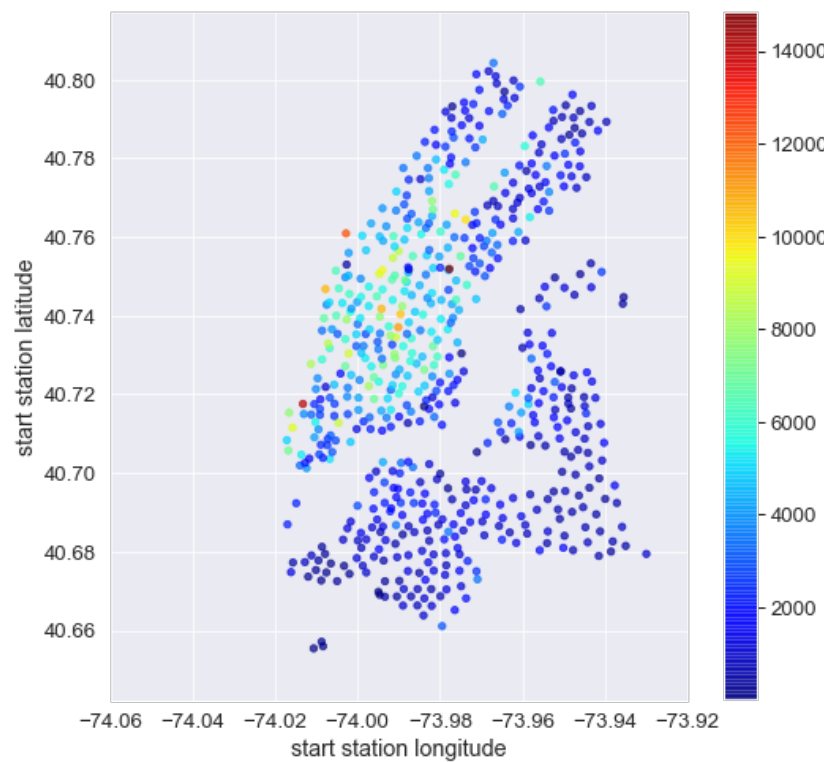
```
# aggregate count by lat-lon pairs
start_lat_lon_counts = data.groupby(['start station latitude',
                                     'start station longitude'])['bikeid'].agg('count').reset_index()
end_lat_lon_counts = data.groupby(['end station latitude',
                                   'end station longitude'])['bikeid'].agg('count').reset_index()
```

```
start_lat_lon_counts.head()
```

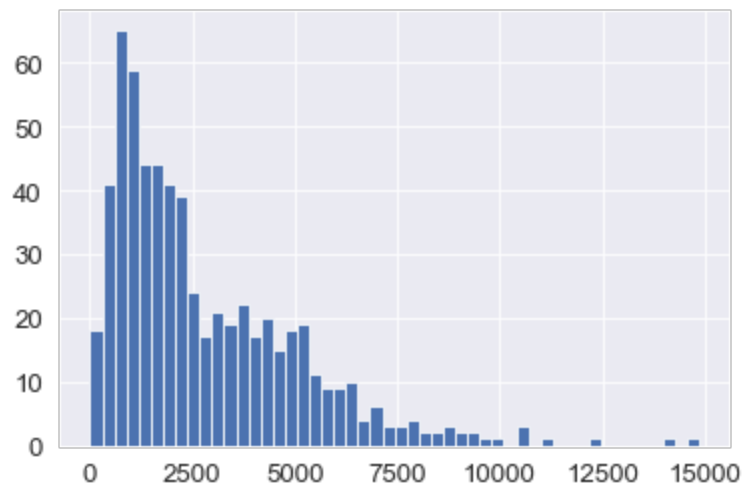
	start station latitude	start station longitude	bikeid
0	40.655400	-74.010628	67
1	40.655949	-74.008359	62
2	40.657089	-74.008702	285
3	40.661063	-73.979453	3028
4	40.663779	-73.983968	1055

```
f, (ax1,ax2) = plt.subplots(1,2,figsize=(18,8))

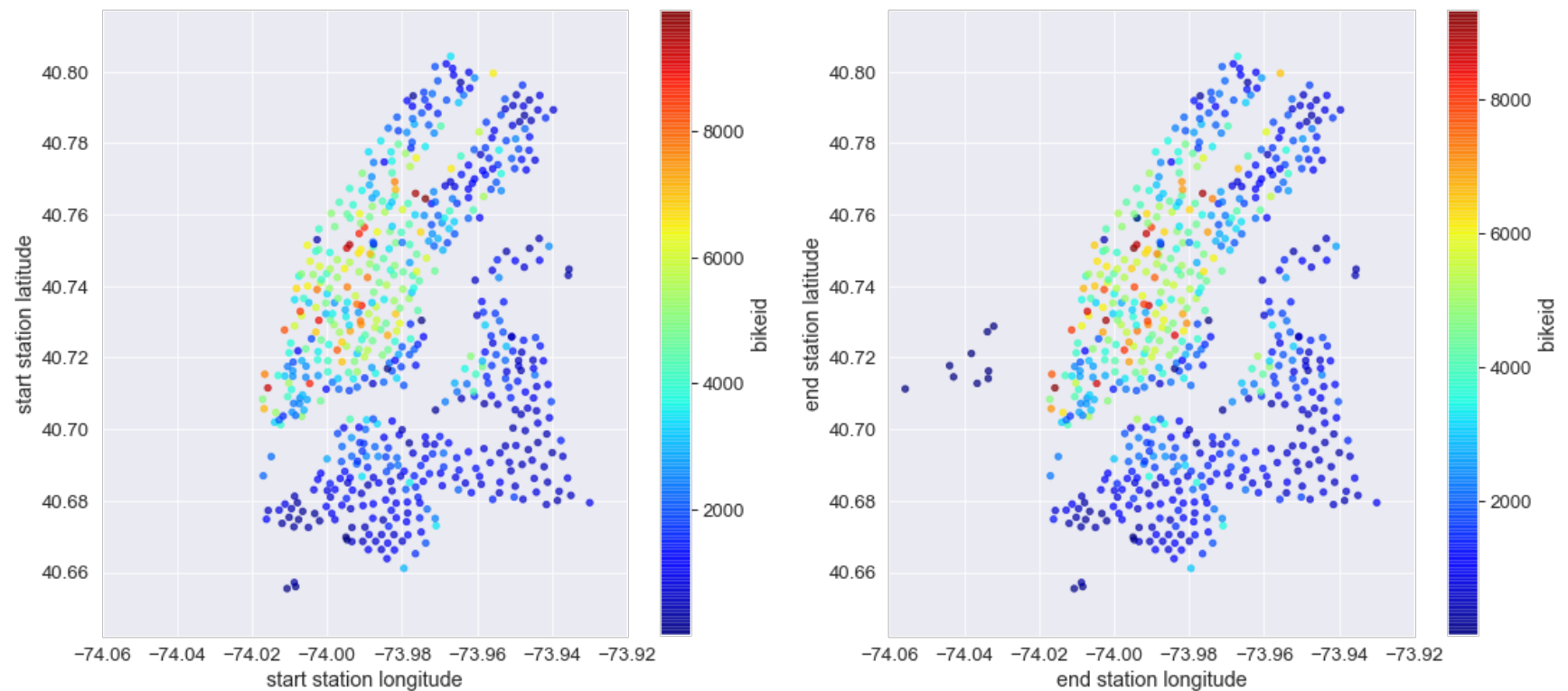
start_lat_lon_counts.plot(kind="scatter", x="start station longitude", y="start station latitude", a
c="bikeid", cmap=plt.get_cmap("jet"), colorbar=True,ax=ax1, xlim=(-74.06, -73.92));
end_lat_lon_counts.plot(kind="scatter", x="end station longitude", y="end station latitude", alpha=0
c="bikeid", cmap=plt.get_cmap("jet"), colorbar=True,ax=ax2, xlim=(-74.06, -73.92));
```



```
start_lat_lon_counts.bikeid.hist(bins=50);
```

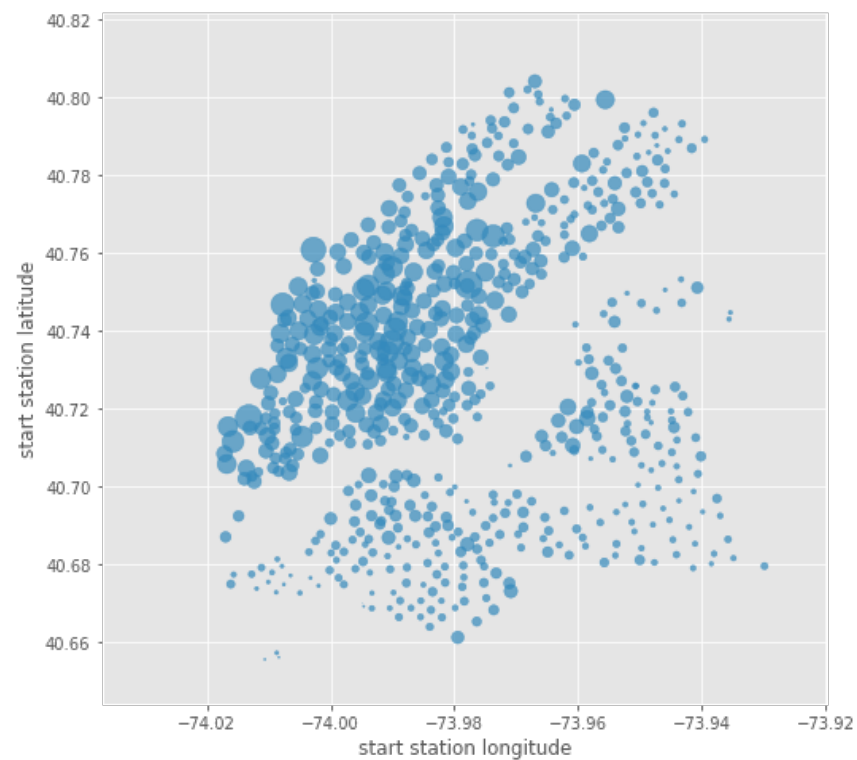
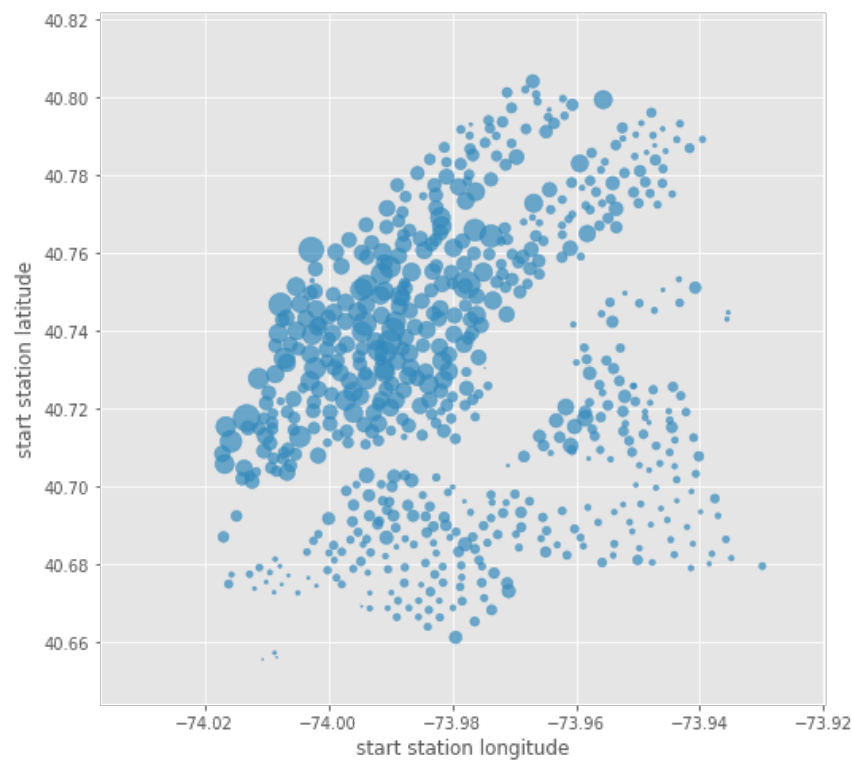


```
_, (ax1,ax2) = plt.subplots(1,2,figsize=(18,8))
# plot derjenigen Stations, die nicht übermäßig frequentiert werden
start_lat_lon_counts[start_lat_lon_counts.bikeid<10000].plot(kind="scatter", x="start station longitude",
    c="bikeid", cmap=plt.get_cmap("jet"), colorbar=True,ax=ax1, xlim=(-74.06, -73.92))
end_lat_lon_counts[end_lat_lon_counts.bikeid<10000].plot(kind="scatter", x="end station longitude",
    c="bikeid", cmap=plt.get_cmap("jet"), colorbar=True,ax=ax2, xlim=(-74.06, -73.92));
```



```
# Darstellung der Häufigkeit mit Größe der Kreise statt mit Farbe
_, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 8))

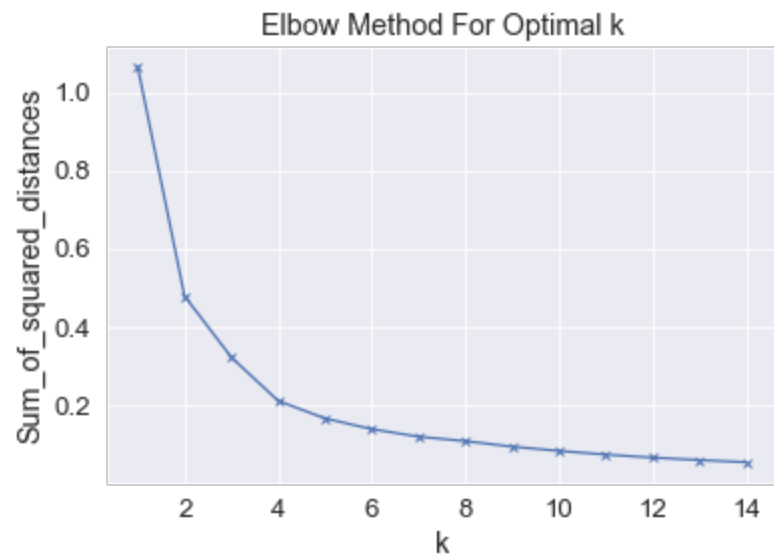
start_lat_lon_counts.plot(kind="scatter", x="start station longitude", y="start station latitude", a
                           , s=start_lat_lon_counts["bikeid"]/50, ax=ax1)
start_lat_lon_counts.plot(kind="scatter", x="start station longitude", y="start station latitude", a
                           s=start_lat_lon_counts["bikeid"]/50, ax=ax2);
```



Clustering der start stations

```
cluster_cols = ['start station latitude', 'start station longitude']  
cl_data = start_lat_lon_counts[cluster_cols].copy()
```

```
sosd = compute_sosd(cl_data, range(1,15))  
  
plt.plot(range(1,15), sosd, 'bx-')  
plt.xlabel('k')  
plt.ylabel('Sum_of_squared_distances')  
plt.title('Elbow Method For Optimal k')  
plt.show()
```



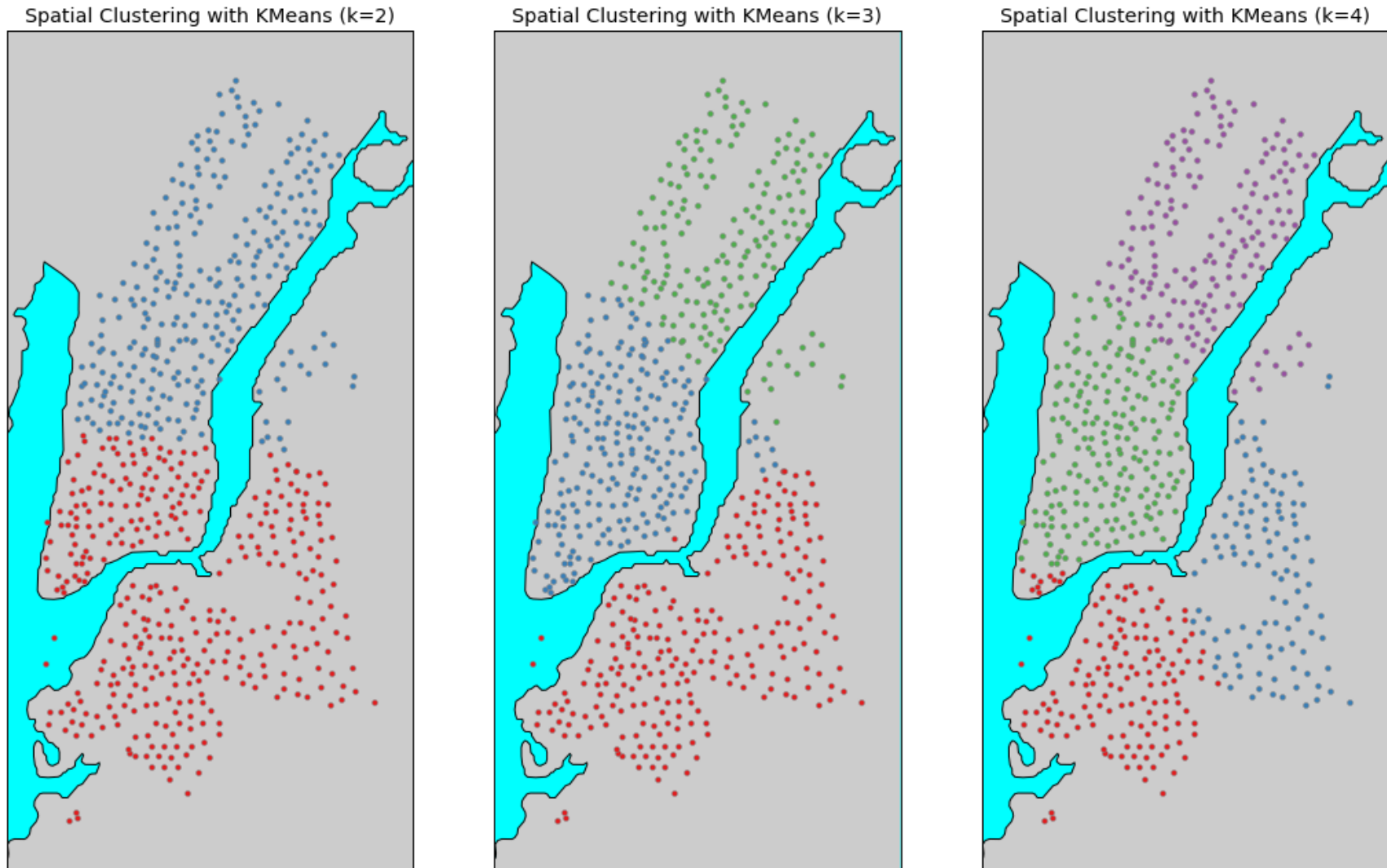
Wir sehen keinen klaren 'elbow' hier, aber $k = 2, 3$, oder 4 scheinen am besten geeignet zu sein.

```
k = [2,3,4]
n = len(k)
plt.subplots(nrows=1, ncols=3, figsize=(18,15))

for ind, i in enumerate(k):
    est = KMeans(n_clusters=i).fit(cl_data)
    start_lat_lon_counts['cluster'] = est.predict(cl_data).tolist()

    ax = plt.subplot(1, 3, ind+1)
    ax.set_title("Spatial Clustering with KMeans (k={})".format(i))

    plot_stations_map(ax, start_lat_lon_counts, 'cluster',
                      'start station latitude', 'start station longitude')
```



$k = 3$ oder 4 erklären die Lage der Stationen besser als $k = 2$. In keinem der Clusterings sind alle Cluster jedoch gut voneinander getrennt.

DBSCAN

```
cluster_cols = ['start station latitude', 'start station longitude']
cl_data = start_lat_lon_counts[cluster_cols].copy()
```

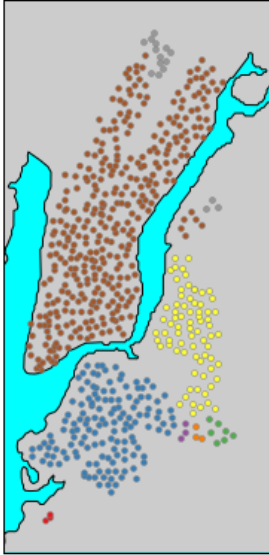
```
eps = [0.004, 0.005] # unit: latitude/longitude
min_sample = [3, 4, 5]
n1, n2 = len(eps), len(min_sample)
plt.subplots(nrows=n1, ncols=n2, figsize=(20, 15))

for i in range(n1):
    for j in range(n2):
        est = DBSCAN(eps=eps[i], min_samples=min_sample[j], metric="euclidean").fit(cl_data)
        start_lat_lon_counts['cluster'] = est.labels_.tolist()

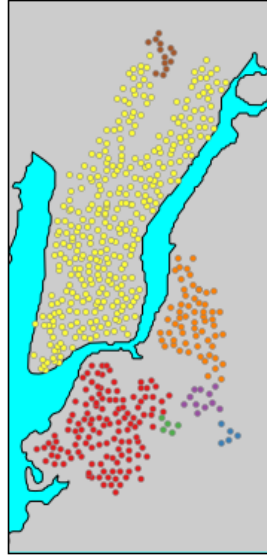
        ax = plt.subplot(n1, n2, n2*i+j+1)
        ax.set_title("DBSCAN ('euclidean', eps={}, min_sample={})".format(eps[i], min_sample[j]))

        plot_stations_map(ax, start_lat_lon_counts, 'cluster',
                           'start station latitude', 'start station longitude')
```

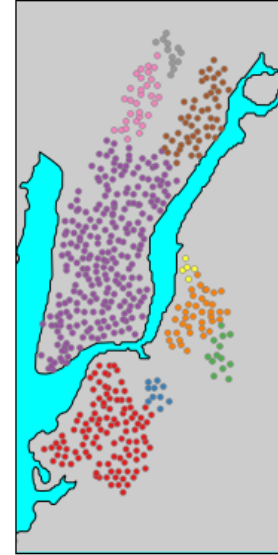
DBSCAN ('euclidean', eps=0.004, min_sample=3)



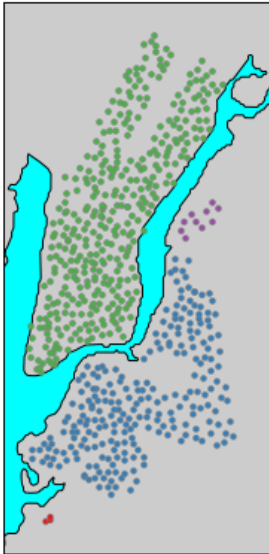
DBSCAN ('euclidean', eps=0.004, min_sample=4)



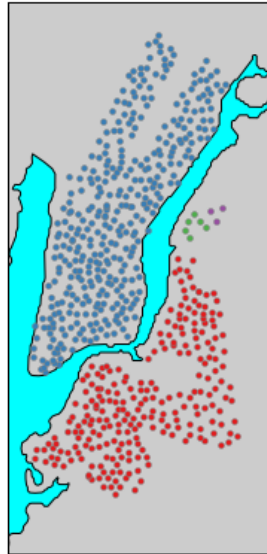
DBSCAN ('euclidean', eps=0.004, min_sample=5)



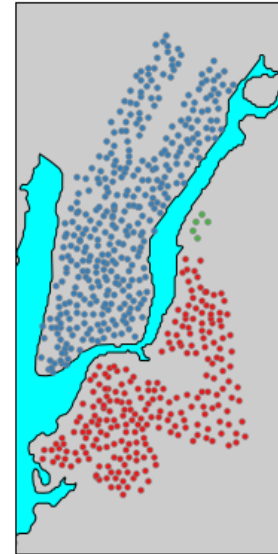
DBSCAN ('euclidean', eps=0.005, min_sample=3)



DBSCAN ('euclidean', eps=0.005, min_sample=4)



DBSCAN ('euclidean', eps=0.005, min_sample=5)



- Nur Kernpunkte sind dargestellt, Outlier wurden nicht eingezeichnet.
- Mit $\text{eps} = 0.04$ ist der Radius so klein, dass die Cluster viel zu klein werden.

- Für den größeren Wert von epsilon sind die Ergebnisse sehr ähnlich.

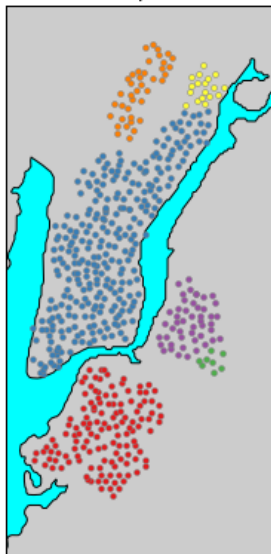
Great Circle Distance

```
def greatCircleDistance(x, y):  
    lat1, lon1 = x[0], x[1]  
    lat2, lon2 = y[0], y[1]  
    return vincenty((lat1, lon1), (lat2, lon2)).meters
```

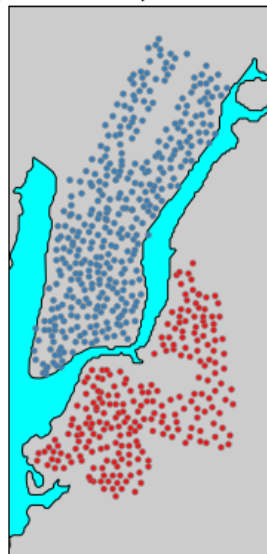
Wertebereich für `eps` und `min_sample` angepasst an neue Einheiten (Einheit: Meter): `eps` von 500 bis 700 und `min_sample` von 5 bis 10.

```
eps = [500, 600, 700]    # unit: meter  
min_sample = [8, 10]  
n1, n2 = len(eps), len(min_sample)  
plt.subplots(nrows=n2, ncols=n1, figsize=(20, 15))  
  
for j in range(n2):  
    for i in range(n1):  
        est = DBSCAN(eps=eps[i], min_samples=min_sample[j], metric=greatCircleDistance).fit(cl_data)  
        start_lat_lon_counts['cluster'] = est.labels_.tolist()  
  
        ax = plt.subplot(n2, n1, n1*j+i+1)  
        ax.set_title("DBSCAN ('greatCircle', eps={}, min_sample={})".format(eps[i], min_sample[j]))  
  
        plot_stations_map(ax, start_lat_lon_counts, 'cluster',  
                          'start station latitude', 'start station longitude')
```

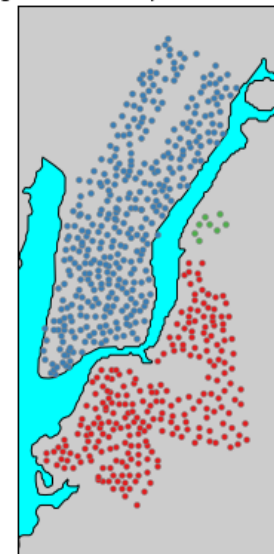
DBSCAN ('greatCircle', eps=500, min_sample=8)



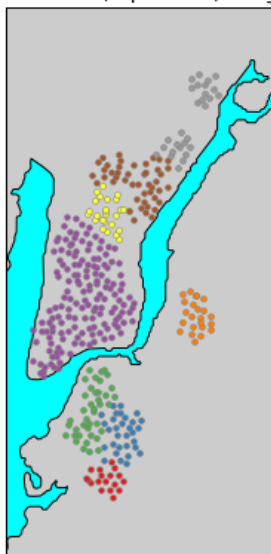
DBSCAN ('greatCircle', eps=600, min_sample=8)



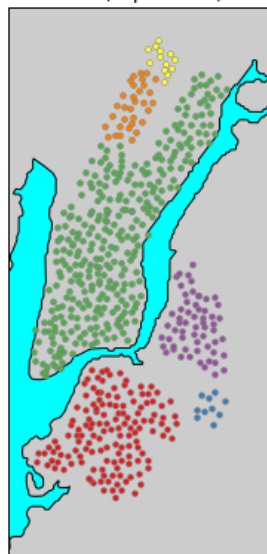
DBSCAN ('greatCircle', eps=700, min_sample=8)



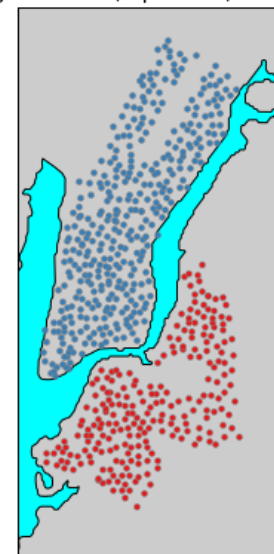
DBSCAN ('greatCircle', eps=500, min_sample=10)



DBSCAN ('greatCircle', eps=600, min_sample=10)



DBSCAN ('greatCircle', eps=700, min_sample=10)



- Clustering-Ergebnisse besser mit der Great Circle Distanz. Nicht überraschend, da die Länge der Latitude-Einheit nicht gleich der Länge der Longitude-Einheit ist.

- Clustering-Ergebnisse besser zu deuten, v.a. bei den beiden Clustern mit kleinerem Radius (`eps`=500): pro Cluster haben alle Punkte in Umgebung von 500 Metern 8 bzw. 10 Nachbarstationen, d.h. dies sind Gegenden, in denen der Anbieter viel Traffic erwartet. Es handelt sich hierbei also vermutlich um dicht besiedelte Gegenden oder vielleicht Arbeitsplatz-Zentren. Hierfür wäre es nun interessant, eine passende Datenquelle zu verknüpfen.

Vergleich K -Means & DBSCAN

- Mit DBSCAN werden manche Stationen zu Outliern und manche Cluster werden sehr klein. Mit K -Means werden alle Stationen einem Cluster zugeordnet und die Cluster-Größen sind untereinander ähnlich.
- DBSCAN kann im Gegensatz zu K -Means den Fluß implizit als eine natürlich Grenze erkennen und trennt Stationen auf verschiedenen Seiten des Flusses.
- Mit der Great Circle Distanz bekommt man sinnvollere Cluster.